

IPaC Semantic OS (IPaC смысловая ОС)

Conceptual Architecture Candidate (концептуальный архитектурный кандидат)

Visible Result Review Draft v0.1 / Видимый результат для рассмотрения v0.1

Status (статус): candidate (кандидат), not_canon (не канон), not_decided (решение не принято)

Project (проект): IPaC_NIR_SEMANTIC_OS

artifact_type	review_draft / conceptual_architecture_candidate (черновик для рассмотрения / концептуальный архитектурный кандидат)
canon_status	not_canon (не канон)
decision_status	not_decided (решение не принято)
purpose	показать видимый результат НИР без преждевременной канонизации
source_mode	по текущему Project Supervisor (супервизору проекта) и принятым operational checkpoints (операционным контрольным точкам)
forbidden	не считать финальной архитектурой, не считать canon (каноном), не выполнять Git action (Git-действие) из документа

Документ предназначен для демонстрации результата работы и запуска review (рассмотрения), а не для canonization (канонизации).

1. Executive Summary / краткое резюме

IPaC OS (IPaC смысловая ОС) в этом документе рассматривается как Architecture Candidate (архитектурный кандидат): операционная система для производства, фиксации, маршрутизации, проверки, восстановления и развития смыслов в гибридной среде Human Intelligence + Silicon Intelligence (человеческий интеллект + кремниевый интеллект).

Документ нужен как видимый результат НИР: он показывает, что работа перешла от подготовки процесса к оформлению архитектурного предмета. Это не canon (канон), не final architecture (финальная архитектура) и не architecture decision (архитектурное решение).

Короткая кандидатная формула: IPaC OS (IPaC смысловая ОС) - это смысловая операционная система, в которой Meaning (смысл) становится операционным объектом, а Project Supervisor (супервизор проекта), Obsidian (ассоциативное хранилище Obsidian), Git (система контроля версий Git), Context Working Set (рабочий набор контекста) и будущий Codex + VS Code (Кодекс + VS Code) образуют управляемый контур смыслового производства.

2. Зачем нужен этот документ

Условия реального мира требуют показать результат: не только последовательность спринтов, но и оформленный материал, который можно дать на review (рассмотрение), защитить как результат НИР и использовать как точку дальнейшей разработки.

Поэтому этот документ делает минимально достаточный шаг: даёт обзор архитектурного кандидата, перечисляет ключевые элементы, показывает рабочий цикл и фиксирует границы. Он не закрывает исследование, а создаёт объект, вокруг которого можно вести следующую работу.

3. Статус и границы

Поле	Значение	Смысл
status (статус)	candidate (кандидат)	материал можно рассматривать и развивать
canon_status (статус канона)	not_canon (не канон)	нельзя объявлять финальной рамкой
decision_status (статус решения)	not_decided (решение не принято)	требуется review (рассмотрение) и Human Decision (человеческое решение)
score (границы)	review draft (черновик для рассмотрения)	документ показывает результат, но не заменяет процесс принятия решений

4. Проблема, на которую отвечает IPaC OS (IPaC смысловая ОС)

Современная работа с AI (искусственным интеллектом), знаниями и проектами быстро производит тексты, идеи, гипотезы и решения, но часто не умеет надёжно сохранять их смысловой статус, происхождение, границы применения и траекторию развития.

Обычные инструменты фиксируют документы или сообщения. Но для IPaC важнее не только текст, а Meaning (смысл): что изменилось, какая гипотеза появилась, какое

решение стало возможным, какой риск возник, что нужно сохранить для следующего прохода.

Отсюда кандидатная задача IPaC OS (IPaC смысловой ОС): создать операционный контур, в котором смысл не теряется между тредами, файлами, решениями, Git (системой контроля версий Git), Obsidian (ассоциативным хранилищем Obsidian) и будущими исполнительными инструментами.

5. Архитектурная идея на одной странице

IPaC OS (IPaC смысловая ОС) можно рассматривать как операционную систему управления смыслами. Она не сводится к prompt engineering (промпт-инжинирингу), RAG (генерации с поисковым дополнением), wiki (вики), Obsidian vault (хранилищу Obsidian), software engineering (инженерии ПО) или AI toolchain (цепочке AI-инструментов).

Её предмет шире: производство, фиксация, маршрутизация, проверка, обогащение, восстановление, развитие и возможная канонизация смыслов. Канонизация возможна только после review (рассмотрения), decision (решения) и Human Approval (человеческого одобрения).

5.1 Минимальная карта компонентов

Компонент	Роль	Статус
Meaning (смысл)	операционный объект	candidate (кандидат)
Meaning Logic Layer (слой смысло-логики)	слой операций над смыслами	candidate (кандидат)
Hybrid Processor Node (гибридный процессорный узел)	связка Human Intelligence + Silicon Intelligence (человеческий интеллект + кремниевый интеллект)	candidate (кандидат)
Dual Memory Architecture (двойная архитектура памяти)	Git (система контроля версий Git) фиксирует, Obsidian (ассоциативное хранилище Obsidian) связывает	candidate (кандидат)
Project Supervisor (супервизор проекта)	планировщик исполнения и контроль контекста	operational role (операционная роль)
Interrupt System (система прерываний)	безопасное переключение между задачами и тредами	candidate (кандидат)
Save-Transfer-Restore Area (Область Сохранения - Передачи - Восстановления)	контейнер восстановления смыслового состояния	candidate (кандидат)
External Resource Management (управление внешними ресурсами)	будущий слой действий через Codex + VS Code (Кодекс + VS Code)	candidate (кандидат)
Semantic I/O Layer (смысловой слой ввода-вывода)	преобразование входов в артефакты, решения, трассы и действия	candidate (кандидат)

6. Как это работает: операционный цикл

Минимальный цикл IPaC OS (IPaC смысловой ОС) в кандидатной рамке выглядит так:

- Event (событие) возникает в работе.
- Factography (фактография) фиксирует, что именно произошло.
- Ship Log / Daily Register (Бортовой журнал / Дневной реестр) сохраняет событие дня.
- Interpretation (интерпретация) отделяет смысл события от простой записи.

- Routing (разнесение) определяет, куда событие должно попасть: research (исследование), rule (правило), candidate artifact (артефакт-кандидат), trace (трасса), decision (решение) или parking lot (парковка).
- Artifact (артефакт) создаётся или обновляется в статусе candidate (кандидат), если решение ещё не принято.
- QA Gate (ворота контроля качества) проверяет границы и происхождение.
- Human Decision (человеческое решение) определяет дальнейший статус: review (рассмотрение), decision (решение), canon (канон) или возврат на доработку.

7. Что уже сделано

На текущем этапе получены не только идеи, но и операционные контрольные точки. Это важно для демонстрации прогресса.

- Project Supervisor (супервизор проекта) загружен и прошёл re-entry / recovery (повторный вход / восстановление).
- ARCH-00 Resource Demand (ресурсный запрос ARCH-00) принят как operational checkpoint (операционная контрольная точка).
- ARCH-01 Role & Output Contract (карта роли и контракт выхода ARCH-01) подготовил границы для Conceptual OS Architect (концептуального архитектора ОС).
- ARCH-02 Section 6 Outline Only (только контур раздела 6) подготовил структуру Architecture Candidate Overview (обзора архитектурного кандидата).
- Текущий документ делает видимый review draft (черновик для рассмотрения), чтобы НИР имела предметный результат.

8. Что является результатом уже сейчас

Результат уже сейчас - не финальная операционная система, а управляемая архитектурная рамка и первый видимый документ. Это достаточно, чтобы показать, что тема не заторможена, а перешла от генерации видений к оформлению предмета разработки.

Минимальный предъявляемый результат: есть название, статус, проблема, кандидатная формула, карта компонентов, операционный цикл, границы, риски и ближайшая программа работ.

9. Ближайшая программа работ

Чтобы двигаться быстро и не расползаться, предлагается короткая программа из трёх рабочих шагов.

Шаг	Название	Результат	Граница
1	Review Draft QA (контроль качества черновика)	проверенный документ для показа	не canon (не канон)
2	Section 6 First Fill (первое заполнение раздела 6)	аккуратный overview (обзор)	не заполнять разделы 7-15
3	Operational Demo Plan (план операционной демонстрации)	как показать IPaC OS (IPaC смысловую ОС) в работе	не запускать Codex (Кодекс) без task pack (пакета задач)

10. Риски и защита от провала

Риск	Защита
Риск затягивания процесса	Сократить протокол и показывать review draft (черновик для рассмотрения) как видимый результат.
Риск преждевременной канонизации	Везде держать status: candidate (кандидат), canon_status: not_canon (не канон).
Риск потери контекста	Использовать Context Working Set (рабочий набор контекста) и PAGE_FAULT (отсутствующая страница) вместо догадок.
Риск tool capture (захвата инструментом)	Codex + VS Code (Кодекс + VS Code) считать будущим I/O layer (слоем ввода-вывода), не главным интеллектом.
Риск смешения статусов	Разделять factography (фактографию), interpretation (интерпретацию), decision (решение), canon (канон).

11. Что нельзя делать с этим документом

- Нельзя считать его canon (каноном).
- Нельзя считать его final architecture (финальной архитектурой).
- Нельзя объявлять Куб №1 canon (каноном).
- Нельзя выполнять Git action (Git-действие) на основании самого документа.
- Нельзя утверждать live Git status (живой статус Git), если он не подан отдельной контрольной.
- Нельзя запускать Codex (Кодекс) без отдельного approved task pack (утверждённого пакета задач).

12. Решение, которое нужно от человека

Этот документ можно использовать как предмет Human Review (человеческого рассмотрения). Возможные решения:

- Принять как review draft (черновик для рассмотрения) и показывать как текущий результат НИР.
- Вернуть на точечную редактуру.
- Разрешить следующий Sprint (спринт): Section 6 First Fill (первое заполнение раздела 6).
- Подготовить короткую презентационную версию на 1-2 страницы для внешнего показа.

13. Итоговая формула

IPaC OS (IPaC смысловая ОС) - это Architecture Candidate (архитектурный кандидат) смысловой операционной системы для гибридной вычислительной среды Human Intelligence + Silicon Intelligence (человеческий интеллект + кремниевый интеллект). Её задача - не просто генерировать тексты, а управлять жизненным циклом Meaning (смысла): от события и фактографии до артефакта, проверки, решения и возможной канонизации.

Текущий документ является видимым результатом НИР и точкой перехода к дальнейшей разработке. Он предназначен для review (рассмотрения), а не для canonization (канонизации).